Network Traffic Capture and Analysis Using Wireshark on Ubuntu

Topic: Network Traffic Analysis
Task: Capture and analyze network traffic using Wireshark

---

1. Objective

The objective of this practical was to capture and analyze network traffic generated between Kali Linux and an Ubuntu virtual machine using Wireshark.

The exercise focused on observing four types of network activity:

- TCP three-way handshake
- HTTP GET request
- DNS query
- ICMP Echo Request and Echo Reply

The captured traffic was saved as a PCAP/PCAPNG file for further analysis.

2. Tools and Environment

Component	Purpose
Kali Linux	Generated network requests and initiated connections
Ubuntu Linux	Hosted the Apache web server
Apache2	Provided the HTTP service
Wireshark	Captured and analyzed network packets
curl	Sent HTTP requests
ping	Generated ICMP traffic
nslookup	Generated DNS queries

Lab configuration

- Kali Linux IP address: 192.168.195.128
- Ubuntu IP address: 192.168.195.133
- Web server: Apache2
- HTTP port: TCP 80
- Packet capture interface: eth0
- Capture filename: network-traffic.pcap

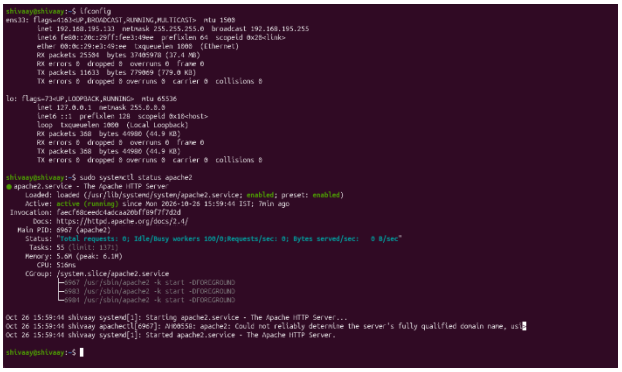
3. Procedure

Step 1: Install and start Apache on Ubuntu

The Apache web server was installed on the Ubuntu virtual machine using:

```
sudo apt update
```

sudo apt install apache2



The service status was checked with:

sudo systemctl status apache2

Apache provided an HTTP endpoint that could be accessed from Kali Linux.

Step 2: Start packet capture

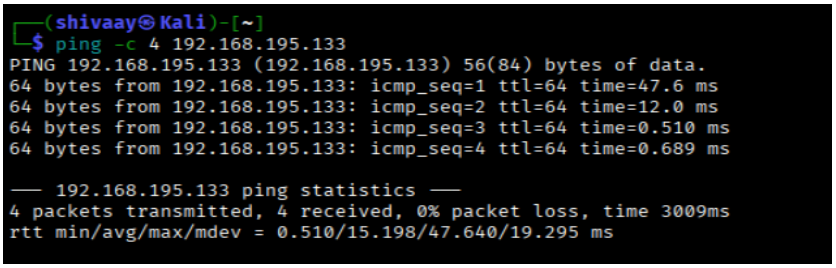
Wireshark was opened on Kali Linux. The network interface connected to the Ubuntu VM was selected, and packet capture was started before generating traffic.

This allowed the packets generated by the subsequent commands to be recorded.

Step 3: Generate ICMP traffic

The following command was executed from Kali:

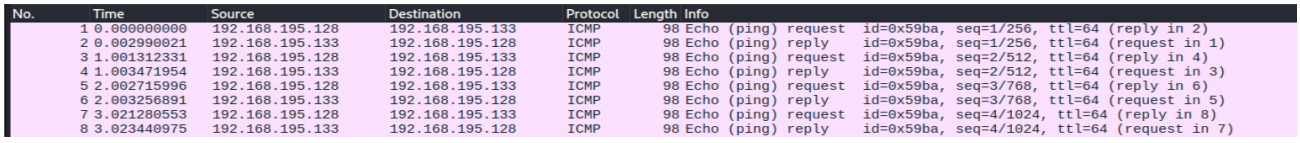
ping -c 4 192.168.195.133



The command sent ICMP Echo Requests to the Ubuntu machine. If permitted by the target and network firewall, the Ubuntu machine returned ICMP Echo Replies.

Wireshark display filter:

icmp

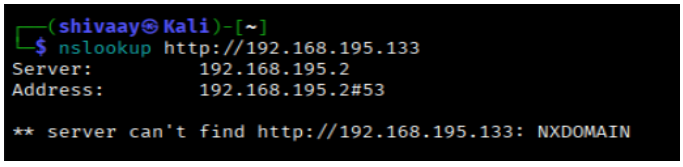


No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.195.128	192.168.195.133	ICMP	98	Echo (ping) request id=0x59ba, seq=1/256, ttl=64 (reply in 2)
2	0.002990021	192.168.195.133	192.168.195.128	ICMP	98	Echo (ping) reply id=0x59ba, seq=1/256, ttl=64 (request in 1)
3	1.001312331	192.168.195.128	192.168.195.133	ICMP	98	Echo (ping) request id=0x59ba, seq=2/512, ttl=64 (reply in 4)
4	1.003471954	192.168.195.133	192.168.195.128	ICMP	98	Echo (ping) reply id=0x59ba, seq=2/512, ttl=64 (request in 3)
5	2.002715996	192.168.195.128	192.168.195.133	ICMP	98	Echo (ping) request id=0x59ba, seq=3/768, ttl=64 (reply in 6)
6	2.003256891	192.168.195.133	192.168.195.128	ICMP	98	Echo (ping) reply id=0x59ba, seq=3/768, ttl=64 (request in 5)
7	3.021280553	192.168.195.128	192.168.195.133	ICMP	98	Echo (ping) request id=0x59ba, seq=4/1024, ttl=64 (reply in 8)
8	3.023440975	192.168.195.133	192.168.195.128	ICMP	98	Echo (ping) reply id=0x59ba, seq=4/1024, ttl=64 (request in 7)

Step 4: Generate DNS traffic

The following command was executed:

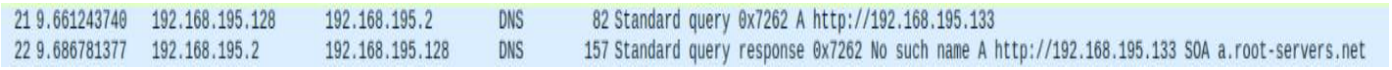
nslookup 192.168.195.133



This generated a DNS query to the configured DNS resolver. Wireshark was used to inspect the DNS request and response.

Wireshark display filter:

dns

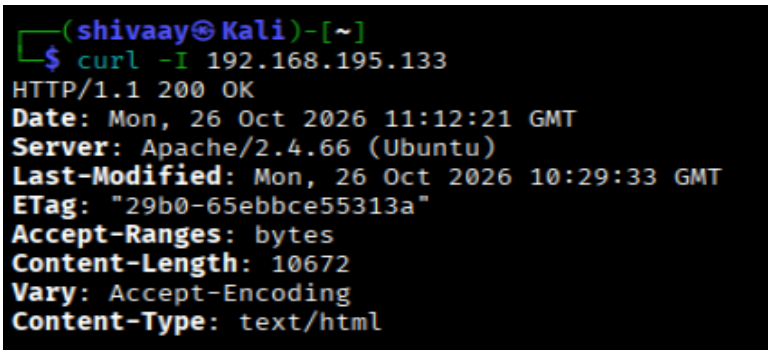


Note: DNS traffic may use UDP port 53 or TCP port 53. The exact packets depend on the resolver and network configuration.

Step 5: Generate HTTP traffic

The Apache web server was accessed from Kali using:

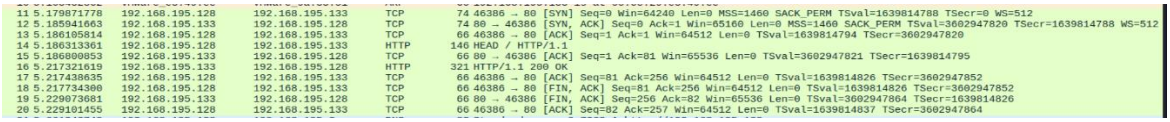
curl -v http://192.168.195.133/



This request generated HTTP traffic to the Ubuntu web server.

Wireshark display filter:

http



The HTTP request was inspected for the request method, requested resource, destination address, and server response. The expected request method was GET.

Step 6: Observe the TCP three-way handshake

When the HTTP connection was established, Wireshark was used to inspect the TCP connection setup.

Wireshark display filter:

tcp.flags.syn == 1

The expected sequence for a new TCP connection was:

a. **SYN:** Kali initiated the TCP connection.
b. **SYN-ACK:** Ubuntu acknowledged the request and indicated readiness to establish the connection.
c. **ACK:** Kali acknowledged the response, completing the three-way handshake.

The TCP handshake establishes a connection before application data is exchanged.

Step 7: Save the packet capture
After the traffic was generated, packet capture was stopped and saved using Wireshark's Save option.

Filename: network-traffic.pcap
The saved capture can be reopened in Wireshark for further investigation.

4. Observations and Analysis

Traffic type	Expected observation	Security relevance
TCP handshake	SYN, SYN-ACK, ACK sequence	Helps identify connection attempts and connection establishment
HTTP	GET request and HTTP response	Unencrypted HTTP may expose request paths, headers, and other transmitted content
DNS	Query and corresponding response, if received	Unexpected domains, unusual query patterns, or suspicious record types may warrant investigation
ICMP	Echo Request and Echo Reply, if allowed	Helps identify connectivity checks and potential network reconnaissance

The actual results should be verified against the saved packet capture. If a response was blocked or a packet was not captured, the expected sequence may be incomplete.

5. Results

The practical demonstrated how to generate common network traffic from Kali Linux and inspect it using Wireshark.

The exercise covered TCP connection establishment, HTTP requests, DNS resolution, and ICMP connectivity testing. The saved packet capture provides evidence for further network traffic analysis and serves as a foundation for understanding Nmap scan traffic and intrusion detection in subsequent tasks.

6. Conclusion

This hands-on exercise provided practical experience with packet capture and protocol analysis. It demonstrated how Wireshark can be used to examine network communication at the packet level and identify the expected behavior of common protocols.

These skills are useful for investigating network connectivity problems, recognizing unusual traffic patterns, and preparing for intrusion detection and security monitoring tasks.